

자동 미분과 경사하강법

코드 해설 b01 ~ b28 · 2권 5장 5.1 ~ 5.13

오늘 볼 코드 28개

묶음	파일	핵심 질문
1. 기울기 재기	b01 감으로 찾기 · b02 한쪽 차분 · b03 양쪽 차분 · b04 편미분	조금 움직이면 얼마나 변하나?
2. 경사하강법	b05 1차원 · b06 골짜기 두 개 · b07 기울기 검산 · b08 NumPy 회귀	기울기 반대로 걸으면 바닥에 갈까?
3. 자동 미분	b09 첫 backward · b10 손과 비교 · b11 규칙 · b12 계산 그래프 · b13 동적 그래프	미분을 PyTorch에게 맡기려면?
4. 연쇄 법칙	b14 손으로 · b15 역전파 한 단계씩	기울기는 왜 거꾸로 흐르나?
5. 함정	b16 쌓임 · b17 zero_ 잊기 · b18 고치기 오류 · b19 기록 끄기 · b20 제자리 연산	자주 하는 실수와 해결법은?
6. 학습률 등	b21~b23 학습률·스케일 · b24 미니배치 · b25~b26 autograd.grad	얼마나, 몇 개씩 보고 고칠까?
7. 실습	b27 당뇨병 미니배치 학습 · b28 정답과 비교	배운 것만으로 실제 데이터 예측

진행 방식

코드마다 ① 결과 예측 → ② 실행 → ③ 한 줄 정리
묶음이 끝나면 확인 문제와 정답 (b27·b28은 scikit-learn 필요)

코드 읽기 전에: 이번 장에 새로 나오는 파이썬 문법

문법	뜻	나오는 곳
`def f(x: float) -> float:`	함수 만들기. ': float'은 입력, '-> float'은 결과 자료형 안내	b02~
`return a, b  /  tuple[float, float]`	값 두 개를 한 번에 돌려주기 (받을 때 a, b = 함수())	b07
수치_미분(f, x)	함수 이름도 값처럼 다른 함수에 넘길 수 있음	b03, b06
`lambda t: (3*t + 1) ** 2`	이름 없는 한 줄 함수	b14
`w -= 0.1 * g`	w = w − 0.1 × g 를 줄여 쓴 것	b08~
`with torch.no_grad():`	이 블록 안의 계산은 기울기 기록을 하지 않음	b17~
`(dy_dx,) = torch.autograd.grad(...)`	결과 묶음(튜플)에서 하나만 꺼내 담기	b25, b26
함수 안에서 자기 자신 호출	재귀: 그래프를 끝까지 따라 내려갈 때	b12
기록[::5]	처음부터 5칸 간격으로 골라내기	b17
`f"{h:<7}"  f"{v:7.2f}"`	<7 왼쪽 정렬 폭 7칸, 7.2f 폭 7칸·소수 둘째 자리	b02, b21
`" ".join(목록)`	글자 목록을 공백으로 이어 붙이기	b21, b22
`np.polyfit(x, y, deg=1)`	직선(1차식)의 정답 w, b를 한 번에 계산	b08, b23

1 기울기 재기

b01 감으로 찾기 · b02 한쪽 차분 · b03 양쪽 차분 · b04 편미분

b01 들어가며: w와 b를 감으로 찾으면?

```
rng = np.random.default_rng(0)              # 난수 생성기 (씨앗 0)
기온 = rng.uniform(18, 34, size=30).round(1)  # 30일치 기온
판매량 = (2.5 * 기온 - 15 + rng.normal(0, 4, size=30)).round()
#         규칙 2.5×기온 - 15 에 잡음을 섞어 만든 예제 데이터

def 평균제곱오차(w: float, b: float) -> float:
    예측 = w * 기온 + b
    return float(((예측 - 판매량) ** 2).mean())

for w, b in [(2.0, 0.0), (2.0, -5.0), (2.5, -10.0), (3.0, -20.0)]:
    print(f"w={w}, b={b:6.1f} → 오차 {평균제곱오차(w, b):7.2f}")
```

```
기온  : [28.2 22.3 18.7 18.3 31. ]
판매량: [51. 40. 31. 33. 63.]
w=2.0, b=   0.0 → 오차   20.76
w=2.0, b=  -5.0 → 오차   39.62
w=2.5, b= -10.0 → 오차   29.22
w=3.0, b= -20.0 → 오차   72.05
```

목표

오차(손실)가 가장 작은 w, b 찾기
직선: 판매량 = w × 기온 + b

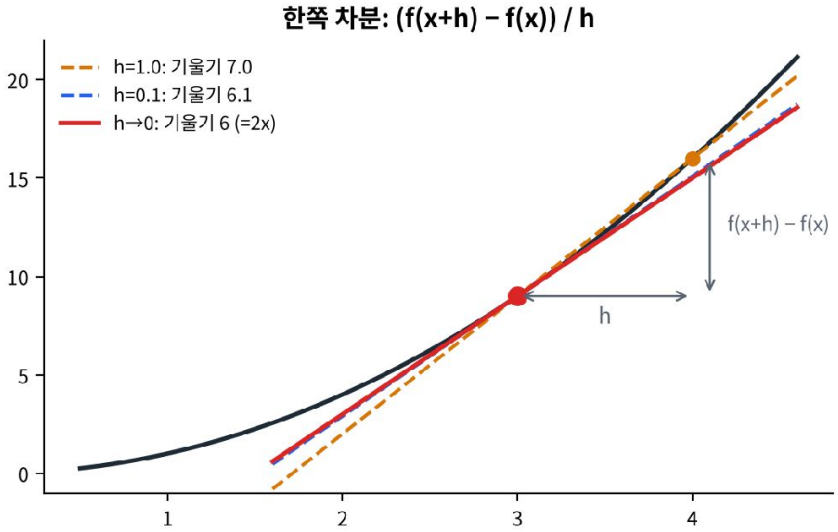
감으로 찾기의 문제

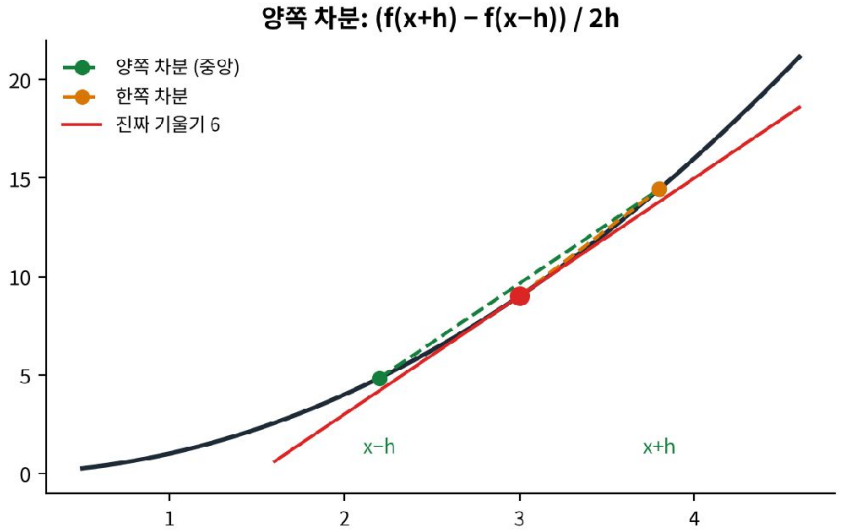
w를 키울까 줄일까?
b는 얼마나 바꿀까?
후보가 무한히 많아서 끝이 없음

안개 낀 산에서 내려오기

주변이 안 보여도 발밑 경사는 느낄 수 있음
→ 내려가는 쪽으로 한 걸음씩
= 기울기를 재서 반대로 이동

개념: 기울기 = 조금 움직였을 때 얼마나 변하나





기울기 읽는 법

\+ 면 오른쪽으로 갈수록 올라감 → 왼쪽(−)으로 가야 내려감
− 면 오른쪽으로 갈수록 내려감 / 0 이면 평평(바닥 후보)

h를 줄이면

h = 1 → 7, 0.1 → 6.1, …
점점 진짜 기울기 6에 가까워짐

개념: 차분(差分) = 차이를 재서 기울기 어림하기

기울기 ≈ 변한 높이 ÷ 움직인 거리 = (f(x + h) − f(x)) ÷ h

말로 풀면

① x를 아주 조금(h) 움직여 봄
② 함수값이 얼마나 변했는지 잼 → 이 '차이'가 차분
③ 변한 양을 움직인 거리 h로 나눔 → 기울기 어림값

예: f(x) = x², x = 3

h = 1 → (16 − 9) ÷ 1 = 7
h = 0.1 → (9.61 − 9) ÷ 0.1 = 6.1
h = 0.01 → (9.0601 − 9) ÷ 0.01 = 6.01
h를 줄일수록 진짜 기울기 6에 가까워짐

이름	식	특징	파일
한쪽 차분	(f(x+h) − f(x)) ÷ h	오른쪽으로만 움직여 봄	b02
양쪽(중앙) 차분	(f(x+h) − f(x−h)) ÷ 2h	양쪽으로 움직여 봄, 더 정확	b03
미분	h → 0 으로 보낸 값	정확한 기울기 (공식으로 구함)	

차분은 공식 없이 함수만 있으면 계산 가능 → '수치 미분'이라고도 부름, 공식 검산에 씀 (b07)

b02 한쪽 차분: h를 줄여 가며 재기

```python
def f(x: float) -> float:
    return x ** 2

x = 3.0
for h in [1.0, 0.1, 0.01, 0.001, 0.0001]:
    기울기 = (f(x + h) - f(x)) / h    # 변한 양 ÷ 움직인 거리
    print(f"h={h:<7} 기울기 ≈ {기울기:.6f}")

for h in [1e-8, 1e-10, 1e-12, 1e-14]:   # 1e-8 = 0.00000001
    print(f"h={h:<7} 기울기 ≈ {(f(x + h) - f(x)) / h:.6f}")
```

```
h=1.0     기울기 ≈ 7.000000
h=0.1     기울기 ≈ 6.100000
h=0.01    기울기 ≈ 6.010000
h=0.001   기울기 ≈ 6.001000
h=0.0001  기울기 ≈ 6.000100

h=1e-08   기울기 ≈ 6.000000
h=1e-10   기울기 ≈ 6.000000
h=1e-12   기울기 ≈ 6.000533
h=1e-14   기울기 ≈ 6.217249  ← 오히려 틀어짐
```

손으로 계산

f(3.1) − f(3) = 9.61 − 9 = 0.61
0.61 ÷ 0.1 = 6.1
남는 0.1 = h 만큼의 오차

h가 너무 작으면?

컴퓨터 실수는 근삿값 (4장 0.1 + 0.2)
아주 작은 차이를 아주 작은 h로 나누면
반올림 오차가 커짐 → 1e−14에서 6.217

그래서

적당한 h (1e−5 정도)를 쓰거나
양쪽 차분(b03)을 씀

b03 양쪽(중앙) 차분: 같은 h로 더 정확하게

```
def 수치_미분(함수, x: float, h: float = 1e-5) -> float:
    """x 양쪽으로 h만큼 움직여 본 높이 차이로 기울기를 잰다."""
    return (함수(x + h) - 함수(x - h)) / (2 * h)
#   함수 자체를 '함수'라는 이름으로 받아서 씀

print(f"한쪽만 (h=0.01): {(f(3.0 + 0.01) - f(3.0)) / 0.01:.6f}")
print(f"양쪽    (h=0.01): {수치_미분(f, 3.0, h=0.01):.6f}")
for x in [-2.0, 0.0, 1.0, 3.0, 5.0]:
    print(f"x={x:4}: 잰 기울기 {수치_미분(f, x):8.4f}, 공식 2x = {2 * x:5.1f}")
```

```
한쪽만 움직이기 (h=0.01): 6.010000
양쪽 움직이기   (h=0.01): 6.000000

x=-2.0: 잰 기울기  -4.0000, 공식 2x =  -4.0
x= 0.0: 잰 기울기   0.0000, 공식 2x =   0.0
x= 1.0: 잰 기울기   2.0000, 공식 2x =   2.0
x= 3.0: 잰 기울기   6.0000, 공식 2x =   6.0
x= 5.0: 잰 기울기  10.0000, 공식 2x =  10.0
```

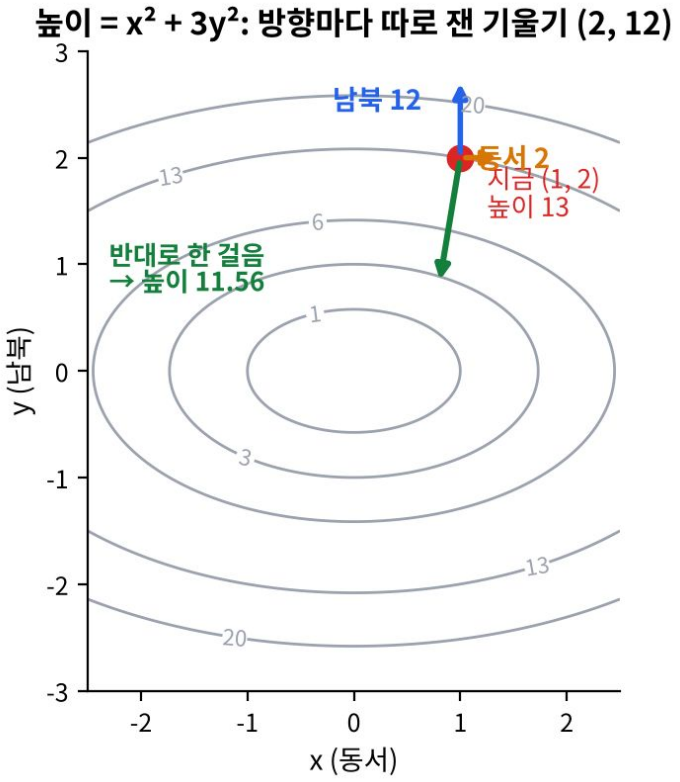
왜 더 정확할까

곡선이 휜 정도가 양쪽에서 반대로 생겨서 서로 지워짐
(그림 오른쪽: 초록 점선이 빨간 접선과 평행)

수치 미분

공식을 몰라도 함수만 있으면 기울기를 잴 수 있음
→ 공식이 맞는지 검산할 때 씀 (b07)

b04 편미분: 방향마다 따로 재기




```python
def 높이(x: float, y: float) -> float:
    return x ** 2 + 3 * y ** 2

h = 1e-5
x, y = 1.0, 2.0
동서 = (높이(x + h, y) - 높이(x - h, y)) / (2 * h)
#       y는 그대로, x만 움직이기
남북 = (높이(x, y + h) - 높이(x, y - h)) / (2 * h)
#       x는 그대로, y만 움직이기
걸음 = 0.01
print(높이(x - 걸음 * 동서, y - 걸음 * 남북))  # 반대로
print(높이(x + 걸음 * 동서, y + 걸음 * 남북))  # 같은 쪽
```

```
동서 방향 기울기: 2.0000  (공식 2x = 2.0)
남북 방향 기울기: 12.0000  (공식 6y = 12.0)
지금 높이: 13.0000
기울기 반대로 한 걸음: 11.5636
기울기 방향으로 한 걸음: 14.5236
```

기울기 = 방향마다 잰 값의 묶음

(2, 12): 남북이 6배 가파름 · 모델의 w, b도 각자 방향 하나씩
반대로 걸으면 내려감(11.56), 같은 쪽이면 올라감(14.52)

확인 문제: 기울기 재기

01 데이터를 만든 규칙 (w=2.5, b=−15)을 넣으면 오차는 0일까?

```
평균제곱오차(2.5, -15.0)
```

02 f(x) = x**3, x = 2에서 h = 0.001이면 기울기는?

```
def f(x): return x ** 3
```

03 f(x) = x**3, x = 3에서 양쪽 차분 기울기는?

```
수치_미분(f, 3.0)
```

04 (x, y) = (2, 1)이면 동서·남북 기울기는?

```
x, y = 2.0, 1.0
```

확인 문제 정답: 기울기 재기

01

15.31 → 0이 아님
판매량에 잡음(표준편차 4)을 섞어 만들어서
$4^2 = 16$ 근처가 최선

02

12.006
공식 $3x^2 = 12$에 가까움 (남는 0.006은 h 때문)

03

27.0000
공식 $3x^2 = 27$과 같음

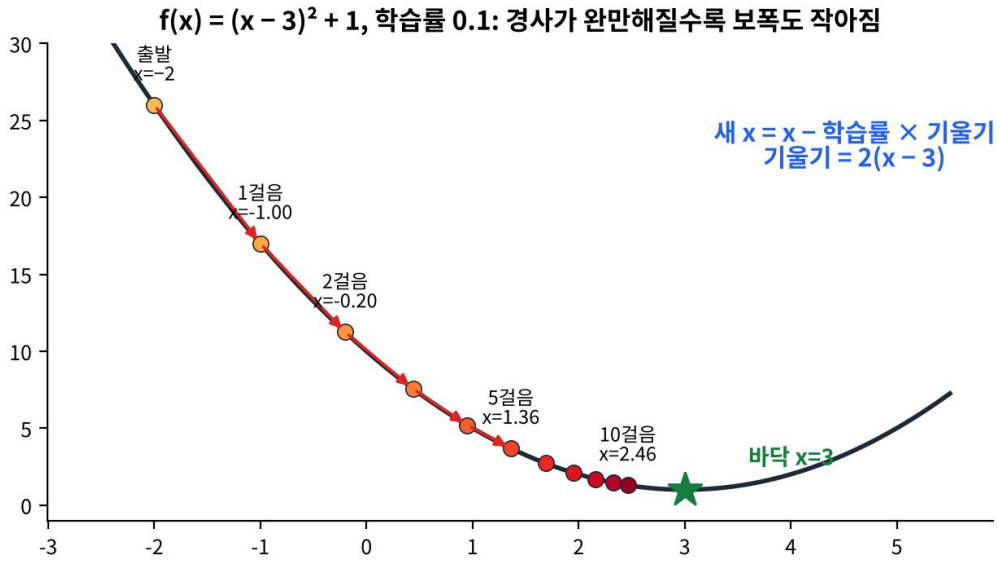
04

동서 4 (= 2x)
남북 6 (= 6y)

2 경사하강법을 손으로

b05 1차원 · b06 골짜기 두 개 · b07 기울기 검산 · b08 NumPy 회귀

b05 경사하강법: 기울기 반대로 조금씩




```python
def f(x: float) -> float:
    return (x - 3) ** 2 + 1    # x=3에서 가장 낮은 1

def 기울기(x: float) -> float:
    return 2 * (x - 3)         # f를 미분한 공식

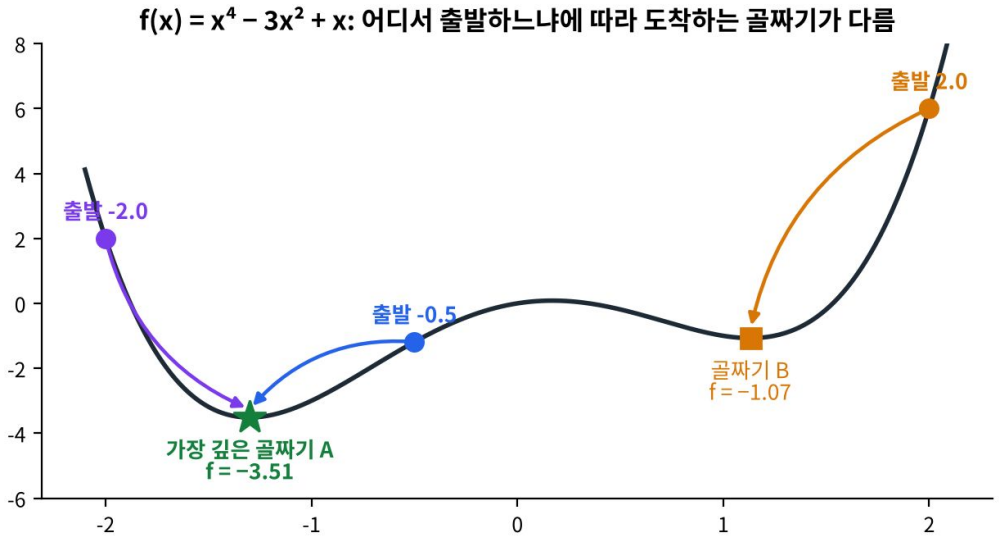
x = -2.0                       # 아무 데서나 출발
학습률 = 0.1                    # 한 번에 걷는 크기
for 걸음 in range(1, 31):
    x = x - 학습률 * 기울기(x)
```

```
 1걸음: x = -1.0000, f = 17.000000
 2걸음: x = -0.2000, f = 11.240000
 3걸음: x =  0.4400, f =  7.553600
 5걸음: x =  1.3616, f =  3.684355
10걸음: x =  2.4631, f =  1.288230
20걸음: x =  2.9424, f =  1.003323
30걸음: x =  2.9938, f =  1.000038
```

손으로 첫 걸음

기울기(−2) = 2 × (−5) = −10
x = −2 − 0.1 × (−10) = −1

b06 골짜기가 두 개면: 출발점이 결과를 정함



실제 신경망: 골짜기가 아주 많지만, 대부분 충분히 깊어서 실용적으로 잘 동작함

```
def f(x: float) -> float:
    return x ** 4 - 3 * x ** 2 + x    # 골짜기 두 개

for 출발 in [2.0, -0.5, -2.0]:
    x = 출발
    for _ in range(200):          # _: 쓰지 않는 반복 변수
        x = x - 0.01 * 수치_미분(f, x)
        # 공식 없이 잰 기울기(b03)만으로 내려가기
```

```
출발  2.0: 도착 x =  1.1309, f(x) =  -1.0702
출발 -0.5: 도착 x = -1.3008, f(x) =  -3.5139
출발 -2.0: 도착 x = -1.3008, f(x) =  -3.5139
```

국소 최솟값 (local minimum)

기울기만 보고 걸으면 가까운 골짜기에 멈춤
출발 2.0은 얕은 골짜기(−1.07)에 갇힘

b07 기울기 검산: 손으로 푼 공식이 맞나

```
x = (기온 - 기온.mean()) / 기온.std()      # 표준화: 평균 0, 표준편차 1
y = 판매량

def 손실(w: float, b: float) -> float:
    return float(((w * x + b - y) ** 2).mean())

def 손_기울기(w: float, b: float) -> tuple[float, float]:
    오차 = w * x + b - y                    # 오차 = 예측 - 정답
    return float((2 * 오차 * x).mean()), float((2 * 오차).mean())

w, b = 0.0, 0.0
잰_w = (손실(w + h, b) - 손실(w - h, b)) / (2 * h)    # 수치 미분
잰_b = (손실(w, b + h) - 손실(w, b - h)) / (2 * h)
공식_w, 공식_b = 손_기울기(w, b)           # 값 두 개 받기
```

```
w 방향: 수치 미분 -25.3119, 공식 -25.3119
b 방향: 수치 미분 -105.0000, 공식 -105.0000
```

표준화한 이유: 기온(18~34)을 그대로 쓰면 학습률 맞추기가 어려움 (b23)

공식 유도

손실 = 평균((wx + b − y)²)
w로 미분: 평균(2 × 오차 × x)
b로 미분: 평균(2 × 오차)

b 기울기 −105

w=0, b=0이면 예측이 모두 0
오차 평균 = −판매량 평균 = −52.5
2 × (−52.5) = −105

b08 NumPy로 선형 회귀 학습

```
w, b = 0.0, 0.0
학습률 = 0.1
for 반복 in range(1, 101):
    오차 = w * x + b - y
    손실 = (오차 ** 2).mean()
    기울기_w = (2 * 오차 * x).mean()       # b07 공식
    기울기_b = (2 * 오차).mean()
    w -= 학습률 * 기울기_w                 # w = w - 학습률 × 기울기
    b -= 학습률 * 기울기_b

정답_w, 정답_b = np.polyfit(x, y, deg=1)  # 한 번에 푸는 공식
print(f"기온 1°C당 판매량 변화: {w / 기온.std():.3f}개")
```

```
  1번째: 손실 2930.500, w = 2.5312, b = 10.5000
  5번째: 손실  503.372, w = 8.5088, b = 35.2968
 10번째: 손실   66.615, w = 11.2970, b = 46.8629
 20번째: 손실   14.683, w = 12.5100, b = 51.8947
 50번째: 손실   14.077, w = 12.6558, b = 52.4993
100번째: 손실   14.077, w = 12.6559, b = 52.5000
polyfit 정답: w = 12.6559, b = 52.5000
기온 1°C당 판매량 변화: 2.495개
```

결과 읽기

50번 만에 정답과 거의 같음
손실 14.08 = 잡음 때문에 남는 오차

w = 12.66인데 2.495?

x를 표준화했으므로
'표준편차 1만큼 따뜻해지면 12.66개'
÷ 기온 표준편차 5.07 → 1°C당 2.495개
→ 데이터를 만든 2.5와 거의 같음

다음 단계

공식(미분)을 매번 손으로 구하기는 힘듦
→ PyTorch에게 맡기기

확인 문제: 경사하강법 손으로

05 학습률을 0.5로 바꾸면 1걸음 뒤 x는?

```
학습률 = 0.5
```

06 출발을 0.0, 1.0으로 하면 각각 어디에 도착할까?

```
for 출발 in [0.0, 1.0]:
```

07 w = 2, b = 50에서 검산하면?

```
w, b = 2.0, 50.0
```

08 학습률을 0.5로 하면 손실이 어떻게 줄까?

```
학습률 = 0.5
```

확인 문제 정답: 경사하강법 손으로

05

x = 3.0 (바로 바닥)
−2 − 0.5 × (−10) = 3
이 함수에서는 0.5가 딱 한 걸음 크기

06

0.0 → x = −1.3008 (깊은 골짜기)
1.0 → x = 1.1309 (얕은 골짜기)
가운데 봉우리(0.17 근처)의 어느 쪽에서 출발하느냐

07

w: −21.3119 / b: −5.0000
수치 미분과 공식이 둘 다 일치

08

1번째 2930.5 → 2번째부터 14.077
한 번 고치고 바로 정답 (w 12.6559, b 52.5)
표준화한 x라서 0.5가 정확한 보폭

3

PyTorch 자동 미분

b09 첫 backward · b10 손과 비교 · b11 규칙 · b12 계산 그래프 · b13 동적 그래프

b09 첫 backward: 미분을 맡기는 세 단계

```
import torch

x = torch.tensor(3.0, requires_grad=True)    # ① 기울기 필요 표시
y = x ** 2                                   # ② 평소처럼 계산
print(y)

y.backward()                                 # ③ y를 x로 미분해 줘
print("PyTorch가 구한 기울기:", x.grad)       # 결과는 x.grad에
print("손으로 구한 기울기 2x:", 2 * 3.0)
h = 1e-3
print("수치 미분:", ((3.0 + h) ** 2 - (3.0 - h) ** 2) / (2 * h))
```

```
tensor(9., grad_fn=<PowBackward0>)
PyTorch가 구한 기울기: tensor(6.)
손으로 구한 기울기 2x: 6.0
수치 미분: 5.999999999999339
```

코드	뜻
requires_grad=True	이 값으로 미분할 거라 계산을 기록해 둬
grad_fn=<PowBackward0>	y가 '제곱'으로 만들어졌다는 기록
y.backward()	기록을 거꾸로 따라가며 미분
x.grad	dy/dx 결과가 저장되는 곳

공식이 필요 없음

손 미분(b07)도 수치 미분(b03)도 없이 정확한 6을 바로 얻음

b10 손으로 구한 기울기와 같을까

```
# 손으로 (NumPy, w=0, b=0)
오차 = 0.0 * x_np + 0.0 - y_np
손_w, 손_b = (2 * 오차 * x_np).mean(), (2 * 오차).mean()

# PyTorch에게 맡기기
x = torch.from_numpy(x_np)                # float64 그대로 (4장 b05)
y = torch.from_numpy(y_np)
w = torch.tensor(0.0, dtype=torch.float64, requires_grad=True)
b = torch.tensor(0.0, dtype=torch.float64, requires_grad=True)
손실 = ((w * x + b - y) ** 2).mean()        # 공식 없이 손실만 계산
손실.backward()
print(w.grad.item(), b.grad.item())
```

```
손으로  : w 기울기 -25.3118789700, b 기울기 -105.0000000000
PyTorch : w 기울기 -25.3118789700, b 기울기 -105.0000000000
같은가? True True
```

핵심

손실만 계산하면 w, b 기울기가 한 번에 나옴
w, b가 몇 개든 똑같이 동작 (b27은 11개)

float64를 쓴 이유

NumPy 결과와 소수 10자리까지 비교하려고
평소 학습은 float32

b11 backward의 규칙 세 가지

```
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
y = x ** 2                          # 결과가 숫자 3개
try:
    y.backward()
except RuntimeError as e:
    print("오류 1:", e)

y.sum().backward()                  # 하나로 모으면 됨
print("x.grad:", x.grad)            # 각 칸의 기울기 2x

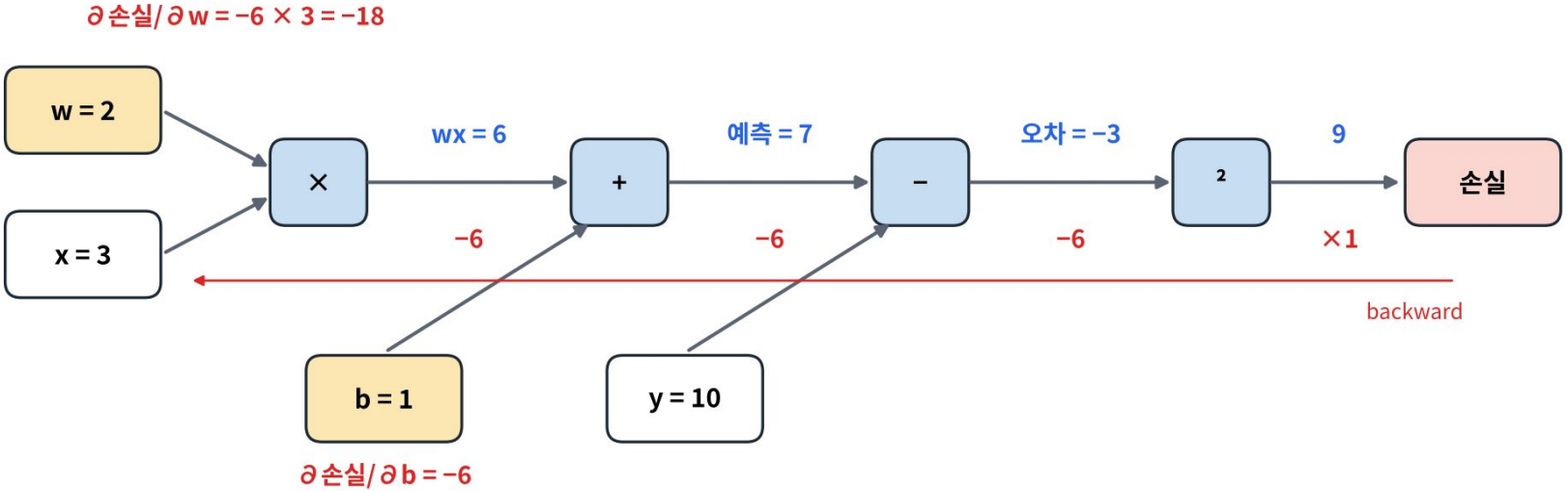
try:
    torch.tensor([1, 2, 3], requires_grad=True)    # 정수
except RuntimeError as e:
    print("오류 2:", e)
print("x는 잎인가?", x.is_leaf, "/ y는 잎인가?", y.is_leaf)
print("y.grad:", y.grad)
```

```
오류 1: grad can be implicitly created only for
  scalar outputs
x.grad: tensor([2., 4., 6.])
오류 2: Only Tensors of floating point and complex
  dtype can require gradients
x는 잎인가? True / y는 잎인가? False
y.grad: None  (+경고 메시지)
```

규칙	이유와 해결
① backward는 숫자 하나에서	손실은 원래 숫자 하나 벡터면 .sum() / .mean()으로 모으기
② 실수만 requires_grad	정수는 '조금 움직이기'가 안 됨 → 1.0처럼 실수로
③ 기울기는 잎에만 저장	잎 = 직접 만든 텐서 (x, w, b) 중간 결과 y.grad는 None 필요하면 retain_grad() (b15)

sum의 기울기는 모든 칸에 1 → 각 칸의 2x가 그대로 나옴

개념: 계산 그래프 — 앞으로 계산하며 기록, 거꾸로 미분



PyTorch가 하는 일

앞으로 계산할 때: 어떤 연산을 했는지와 중간값을 기록 (grad_fn)
backward 할 때: 기록을 끝에서부터 거꾸로 따라가며 기울기를 곱해 전달

b12 계산 그래프 직접 따라가 보기

```
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
x = torch.tensor(3.0)          # 데이터: 기울기 필요 없음
y = torch.tensor(10.0)
예측 = w * x + b
손실 = (예측 - y) ** 2

def 그래프_따라가기(노드, 깊이: int = 0) -> None:
    if 노드 is None:
        return
    이름 = type(노드).__name__            # 노드 종류 이름
    if 이름 == "AccumulateGrad":          # 잎에 도착
        이름 += f"  ← 잎 텐서 값 {노드.variable.item()}"
    print("    " * 깊이 + 이름)           # 깊이만큼 들여쓰기
    for 다음, _ in 노드.next_functions:
        그래프_따라가기(다음, 깊이 + 1)    # 재귀: 한 칸 더 거꾸로

그래프_따라가기(손실.grad_fn)
```

```
손실: tensor(9., grad_fn=<PowBackward0>)
PowBackward0
    SubBackward0
        AddBackward0
            MulBackward0
                AccumulateGrad  ←잎 텐서 값 2.0
            AccumulateGrad  ←잎 텐서 값 1.0
```

읽는 법

아래로 갈수록 계산의 앞쪽
Pow(²) ← Sub(−) ← Add(+) ← Mul(×)
AccumulateGrad = 잎 w, b에 기울기를 쌓는 곳

x, y는 requires_grad가 없어서 그래프에 나오지 않음

b13 동적 그래프: 실행할 때마다 새로 그림

```
def 요금(사용량: torch.Tensor) -> torch.Tensor:
    """100 넘게 쓰면 넘은 만큼은 단가가 두 배인 누진 요금."""
    if 사용량 > 100:                    # 평범한 파이썬 if
        return 100 * 1.0 + (사용량 - 100) * 2.0
    return 사용량 * 1.0

for 값 in [80.0, 130.0]:
    u = torch.tensor(값, requires_grad=True)
    요금(u).backward()
    print(f"사용량 {값}: 요금 기울기 {u.grad.item()}")

u = torch.tensor(130.0, requires_grad=True)
결과 = 요금(u)
결과.backward()
try:
    결과.backward()                     # 같은 그래프로 한 번 더?
except RuntimeError as e:
    print("오류:", str(e).split(".")[0])
```

```
사용량 80.0: 요금 기울기 1.0
사용량 130.0: 요금 기울기 2.0
오류: Trying to backward through the graph a
  second time (or directly access saved
  tensors after they have already been freed)
```

동적 그래프

if, for를 그대로 써도 됨
실제로 지나간 길만 기록
80 → 단가 1, 130 → 단가 2

backward는 한 번만

끝나면 중간값 기록을 지움
→ 다시 계산(앞으로)부터
학습 루프에서 매번 손실을 새로 계산하는 이유

확인 문제: PyTorch 자동 미분

09 y = x ** 3, x = 2이면 x.grad는?

```
x = torch.tensor(2.0, requires_grad=True)
```

10 손실을 .mean() 대신 .sum()으로 바꾸면 기울기는?

```
손실 = ((w * x + b - y) ** 2).sum()
```

11 y.sum() 대신 y.mean().backward()이면 x.grad는?

```
y.mean().backward()
```

12 b를 빼고 예측 = w * x 로 하면 그래프는?

```
예측 = w * x
```

13 사용량 100.0이면 기울기는?

```
for 값 in [100.0]:
```

확인 문제 정답: PyTorch 자동 미분

09

tensor(12.)
$3x^2 = 3 \times 4 = 12$

10

30배가 됨: w −759.36, b −3150
데이터 30개를 평균 내지 않고 더했으니까

11

[0.6667, 1.3333, 2.0000]
2x ÷ 3 (평균이라 칸마다 1/3씩)

12

```
PowBackward0
  SubBackward0
    MulBackward0
      AccumulateGrad (w)
```

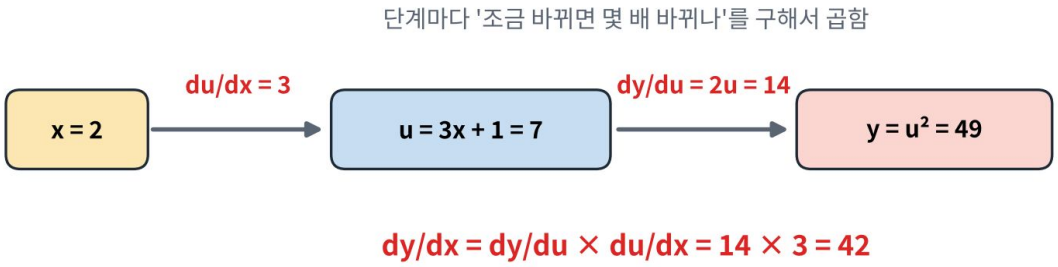
13

1.0
100 > 100은 False라 아래쪽(단가 1) 길로 감

4 연쇄 법칙

b14 손으로 · b15 역전파 한 단계씩

b14 연쇄 법칙: 단계별 기울기를 곱하기




```
# y = (3x + 1)^2 를 두 단계로: u = 3x + 1, y = u^2
x값 = 2.0
u값 = 3 * x값 + 1
du_dx = 3.0              # x가 1 움직이면 u는 3
dy_du = 2 * u값          # u가 1 움직이면 y는 2u
print(f"연쇄 법칙(손): {dy_du} × {du_dx} = {dy_du * du_dx}")

x = torch.tensor(x값, requires_grad=True)
y = (3 * x + 1) ** 2
y.backward()
f = lambda t: (3 * t + 1) ** 2    # 한 줄 함수
```

```
연쇄 법칙(손): dy/dx = 14.0 × 3.0 = 42.0
PyTorch      : dy/dx = 42.0
수치 미분    : dy/dx = 42.0000
```

비유

환율 연결: 1달러 = 1300원, 1원 = 0.1엔
→ 1달러 = 1300 × 0.1 엔
변화율도 이렇게 곱해서 연결

b15 역전파를 한 단계씩 손으로

```
x, y = torch.tensor(3.0), torch.tensor(10.0)
w = torch.tensor(2.0, requires_grad=True)
b = torch.tensor(1.0, requires_grad=True)
예측 = w * x + b        # 7
오차 = 예측 - y         # -3
손실 = 오차 ** 2        # 9
예측.retain_grad()      # 중간 결과 기울기도 보관해 줘
오차.retain_grad()
손실.backward()

# 끝에서부터 '지역 기울기'를 곱해 나감
d손실_d오차 = 2 * 오차.item()         # 손실 = 오차^2
d손실_d예측 = d손실_d오차 * 1         # 오차 = 예측 - y
d손실_dw = d손실_d예측 * x.item()     # 예측 = w*x + b
d손실_db = d손실_d예측 * 1
```

```
d손실/d오차: 손으로   -6.0, PyTorch   -6.0
d손실/d예측: 손으로   -6.0, PyTorch   -6.0
d손실/dw: 손으로  -18.0, PyTorch  -18.0
d손실/db: 손으로   -6.0, PyTorch   -6.0
```

단계	지역 기울기	누적
손실 = 오차²	2 × 오차 = −6	−6
오차 = 예측 − y	1	−6 × 1 = −6
예측 = w·x + b → w	x = 3	−6 × 3 = −18
예측 = w·x + b → b	1	−6 × 1 = −6

읽어 보기

w 기울기 −18: w를 키우면 손실이 줄어듦
(예측 7이 정답 10보다 작으니까)
→ 경사하강법은 w를 키우는 쪽으로 이동

확인 문제: 연쇄 법칙

14 x = 1이면 dy/dx는? (손으로 먼저)

```
x값 = 1.0
```

15 정답 y를 10이 아니라 7로 바꾸면 기울기는?

```
x, y = torch.tensor(3.0), torch.tensor(7.0)
```

확인 문제 정답: 연쇄 법칙

14

24
u = 4 → dy/du = 8, du/dx = 3
8 × 3 = 24

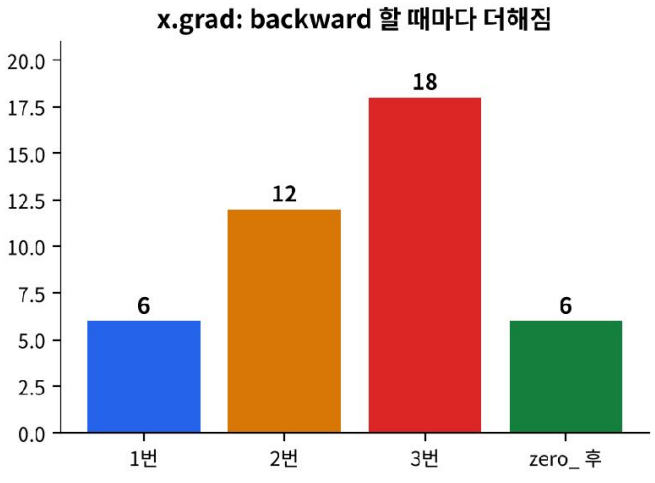
15

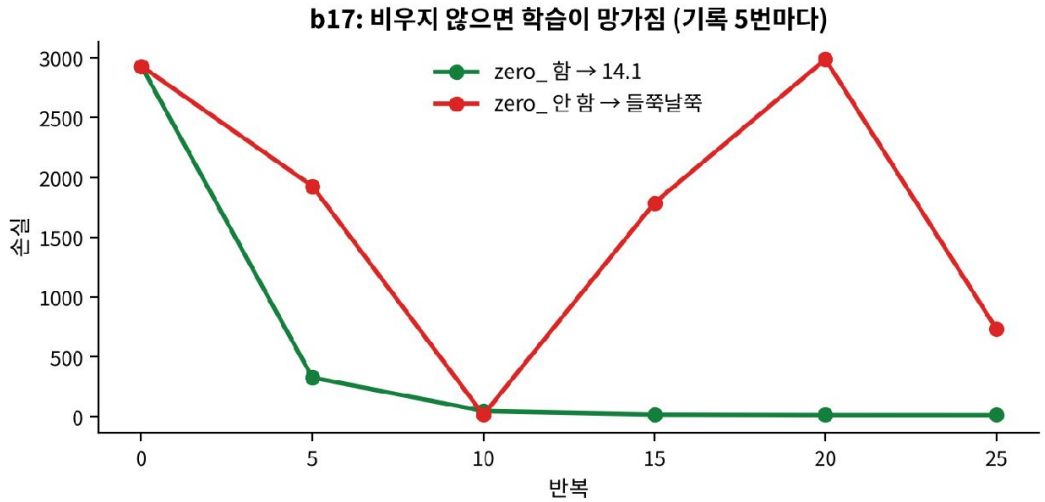
모두 0 (w, b 둘 다)
예측 7 = 정답 → 오차 0 → 고칠 필요 없음

5 자동 미분의 함정

b16 쌓임 · b17 zero_ 잊기 · b18 고치기 오류 · b19 기록 끄기 · b20 제자리 연산

b16 기울기는 덮어쓰지 않고 쌓임






```
x = torch.tensor(3.0, requires_grad=True)
for 번째 in range(1, 4):
    y = x ** 2
    y.backward()
    print(f"{번째}번째 backward 뒤 x.grad = {x.grad.item()}")
x.grad.zero_()              # 비우기 (밑줄 = 제자리)
(x ** 2).backward()
print("비운 뒤 다시:", x.grad.item())
```

```
1번째 backward 뒤 x.grad = 6.0
2번째 backward 뒤 x.grad = 12.0
3번째 backward 뒤 x.grad = 18.0
비운 뒤 다시: 6.0
```

쌓이는 이유: 여러 손실의 기울기를 합칠 때 편하도록 설계됨

b17 zero_를 잊으면 학습이 망가짐

```
def 학습(비우기: bool) -> list[float]:
    w = torch.zeros((), requires_grad=True)  # 숫자 하나짜리 0
    b = torch.zeros((), requires_grad=True)
    기록 = []
    for _ in range(30):
        손실 = ((w * x + b - y) ** 2).mean()
        손실.backward()
        with torch.no_grad():        # 고치기는 기록 안 함 (b18)
            w -= 0.1 * w.grad
            b -= 0.1 * b.grad
        if 비우기:
            w.grad.zero_()
            b.grad.zero_()
        기록.append(손실.item())
    return 기록

for 비우기 in (True, False):
    기록 = 학습(비우기)
    print(..., [round(v, 1) for v in 기록[::5]], "→", round(기록[-1], 1))
```

```
grad 비우기=True : [2930.5, 327.2, 47.7,
  17.7, 14.5, 14.1] → 14.1
grad 비우기=False: [2930.5, 1927.2, 15.8,
  1786.5, 2987.0, 734.2] → 1691.5
```

왜 출렁일까

지난번 기울기까지 더해져서
이미 바닥을 지나쳤는데도 계속 같은 쪽으로 감
→ 관성처럼 넘어갔다 돌아왔다

규칙

backward → 고치기 → zero_
(6장에서는 optimizer.zero_grad())

b18 w를 고칠 때 생기는 오류

```
w = torch.tensor(2.0, requires_grad=True)
손실 = (w * 3.0 - 10.0) ** 2
손실.backward()

try:
    w -= 0.01 * w.grad        # 기록을 켠 채 잎 텐서를 고치면?
except RuntimeError as e:
    print("오류:", e)

w2 = w - 0.01 * w.grad        # 새 텐서를 만들면 오류는 없지만...
print("w2:", w2, "/ 잎인가?", w2.is_leaf)

with torch.no_grad():         # 올바른 방법
    w -= 0.01 * w.grad
print("w:", w, "/ 잎인가?", w.is_leaf)
```

```
오류: a leaf Variable that requires grad is being
  used in an in-place operation.
w2: tensor(2.2400, grad_fn=<SubBackward0>) / 잎인가? False
w: tensor(2.2400, requires_grad=True) / 잎인가? True
```

방법	결과
w -= ...	오류: 고치는 것까지 기록하려다 막힘
w2 = w − ...	w2는 계산 결과(잎 아님) → .grad가 안 쌓임 → 다음 학습 불가
with no_grad(): w -= ...	**w 그대로 잎, 값만 바뀜 → 정답**

기억

'고치기'는 학습할 내용이 아님
→ 기록을 끄고 고침

b19 기록 끄기: no_grad · inference_mode · detach

```
w = torch.tensor([1.0, 2.0], requires_grad=True)
with torch.no_grad():                # 이 안은 기록 안 함
    a = w * 2
print("no_grad       :", a.requires_grad, a.grad_fn)
with torch.inference_mode():         # 예측만 할 때, 더 가벼움
    b = w * 2
print("inference_mode:", b.requires_grad, b.is_inference())
c = w.detach()                       # 기록에서 떼어 낸 '창'
print("detach        :", c.requires_grad,
      c.data_ptr() == w.data_ptr())   # 같은 메모리?
c[0] = 100.0                         # 떼어 낸 쪽을 바꾸면?
print("w도 바뀌었다  :", w)
try:
    w.numpy()
except RuntimeError as e:
    print("오류:", e)
print("detach 후 numpy:", w.detach().numpy())
```

```
no_grad       : False None
inference_mode: False /추론 전용 텐서인가? True
detach        : False /메모리 공유: True
w도 바뀌었다  : tensor([100., 2.], requires_grad=True)
오류: Can't call numpy() on Tensor that requires grad.
  Use tensor.detach().numpy() instead.
detach 후 numpy: [100.   2.]
```

방법	언제
with no_grad():	w 고치기, 평가 (6장)
with inference_mode():	예측만 할 때 (가장 가벼움) 결과로 다시 학습 불가
t.detach()	텐서 하나만 기록에서 떼기 numpy·그래프 그리기 전에 메모리 공유 (4장 b11)

주의

detach는 복사가 아님
바꾸면 w도 바뀜 → 따로 쓰려면 .clone()

b20 제자리 연산이 자동 미분과 부딪힐 때

```
a = torch.tensor([0.0, 1.0, 2.0], requires_grad=True)
b = a.exp()             # exp의 기울기 = 자기 결과(b)를 다시 씀
print("b의 버전:", b._version)
b.add_(1)               # 결과를 제자리에서 바꾸면?
print("b의 버전:", b._version)
try:
    b.sum().backward()
except RuntimeError as e:
    print("오류:", str(e).split(" Hint")[0])

b = a.exp(); c = b + 1    # 밑줄 없이: 새 텐서
c.sum().backward(); print("밑줄 없이:", a.grad)

b = a * 3; b.add_(1)      # ×3의 기울기는 b 값과 무관
b.sum().backward(); print("상관없는 경우:", a.grad)
```

```
b의 버전: 0
b의 버전: 1
오류: one of the variables needed for gradient computation
  has been modified by an inplace operation: ... output 0
  of Exp, is at version 1; expected version 0 instead.
밑줄 없이: tensor([1.0000, 2.7183, 7.3891])
상관없는 경우: tensor([3., 3., 3.])
```

왜 오류?

e^x의 기울기 = e^x 자신
→ backward 때 b 값이 필요한데
add_로 b를 바꿔 버림
버전 번호(_version)로 이를 알아챔

해결

학습 중 계산에서는 밑줄(_) 연산을 피하고
c = b + 1 처럼 새 텐서로
(4장 b19 제자리 연산 주의와 같은 이야기)

확인 문제: 자동 미분의 함정

16 y = x ** 3로 바꾸면 3번 backward 뒤 x.grad는?

```
y = x ** 3
```

17 비우기 False에서 학습률을 0.01로 줄이면?

```
w -= 0.01 * w.grad
```

18 no_grad 안에서 w2 = w − 0.01 * w.grad 를 하면?

```
with torch.no_grad():
    w2 = w - 0.01 * w.grad
```

19 c = w.detach().clone() 후 c[0] = 100 이면 w는?

```
c = w.detach().clone()
```

20 b = a.exp() 대신 b = a + 1 로 하고 add_(1) 하면?

```
b = a + 1
b.add_(1)
```

확인 문제 정답: 자동 미분의 함정

16 81
한 번에 $3x^2 = 27$씩 쌓임: 27 → 54 → 81

17 [2930.5, 1499.9, 35.0, 1012.1, 2779.5, 2349.5] → 781.1
작은 학습률로도 쌓이면 출렁임

18 w2 = 2.24, requires_grad False
기록이 끊긴 새 텐서 → 학습할 값으로 못 씀
w 자체를 고쳐야 함

19 tensor([1., 2.]) 그대로
clone으로 복사했으니 공유 안 함

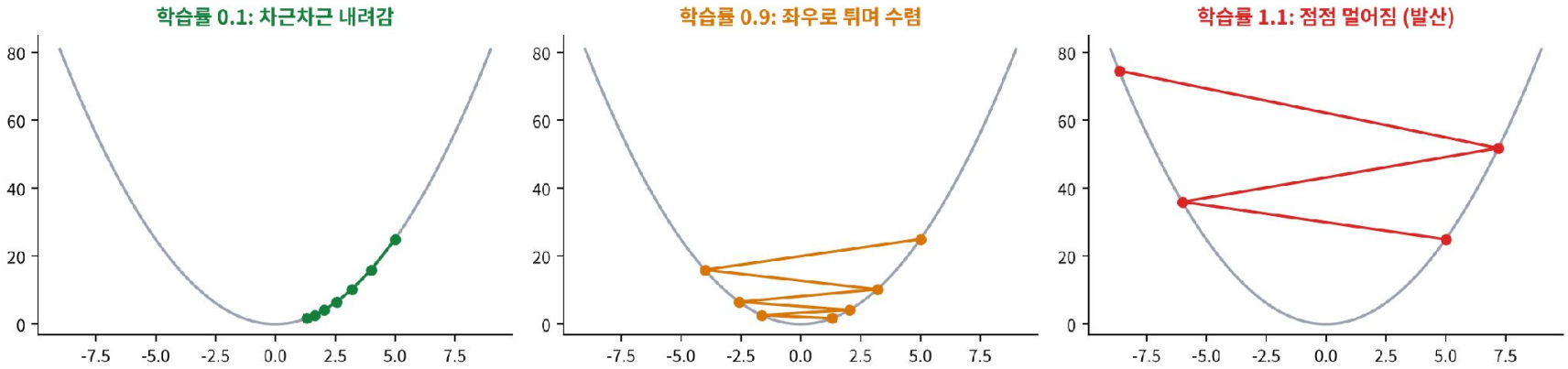
20 오류 없음, a.grad = [1., 1., 1.]
덧셈의 기울기는 b 값이 필요 없어서

6 학습률 · 스케일 · 미니배치

b21~b23 학습률과 스케일 · b24 미니배치 · b25 b26 autograd.grad

b21 학습률: 너무 작으면 느리고, 너무 크면 튐

$f(x) = x^2$, 출발 $x = 5$




```
# f(x) = x^2, 기울기 2x, x=5에서 출발
for 학습률 in [0.01, 0.1, 0.5, 0.9, 1.1]:
    x = 5.0
    자취 = []
    for 걸음 in range(1, 21):
        x = x - 학습률 * 2 * x
        if 걸음 in (1, 2, 3, 10, 20):
            자취.append(f"{x:9.3f}")
```

```
0.01: 4.900  4.802  4.706  4.085   3.338 느림
0.1 : 4.000  3.200  2.560  0.537   0.058
0.5 : 0.000  0.000  0.000  0.000   0.000 한 번에
0.9 :-4.000  3.200 -2.560  0.537   0.058 튀며
1.1 :-6.000  7.200 -8.640 30.959 191.688 발산
```

$x \times (1 - 2 \times$ 학습률$)$: 이 배수의 크기가 1보다 크면 발산

b22 회귀에서 학습률 비교

```
def 학습(x, y, 학습률: float, 횟수: int) -> list[float]:
    w = torch.zeros((), requires_grad=True)
    b = torch.zeros((), requires_grad=True)
    기록 = []
    for 반복 in range(1, 횟수 + 1):
        손실 = ((w * x + b - y) ** 2).mean()
        손실.backward()
        with torch.no_grad():
            w -= 학습률 * w.grad
            b -= 학습률 * b.grad
            w.grad.zero_()
            b.grad.zero_()
        if 반복 in (10, 50, 200):
            기록.append(손실.item())
    return 기록
```

학습률	10번째	50번째	200번째	
0.001	2827.27	2410.94	1328.71	너무 느림
0.01	2041.39	416.8	15.02	아직 덜 내려감
0.1	66.61	14.08	14.08	**적당**
0.9	66.61	14.08	14.08	튀지만 수렴
1.05	16229.2	3.3e+07	8.7e+19	**발산**

가장 많이 하는 실수

손실이 nan, inf, 아주 큰 수 → 학습률부터 줄이기
손실이 거의 안 줄면 → 학습률 키우기

b23 스케일: 표준화하지 않으면 학습률 맞추기가 어려움

```
x_원래 = torch.tensor(기온, dtype=torch.float32)   # 18~34 그대로
for 학습률 in [0.01, 0.001]:
    w = torch.zeros((), requires_grad=True)
    b = torch.zeros((), requires_grad=True)
    for 반복 in range(1, 5001):
        손실 = ((w * x_원래 + b - y) ** 2).mean()
        ...  # backward, no_grad 고치기, zero_ (b22와 같음)
```

```
학습률 0.01,     5번째: 손실 3.484e+12, w = 940409.562, b = 34166.523
학습률 0.01,   100번째: 손실 nan, w = nan, b = nan
학습률 0.01,  5000번째: 손실 nan, w = nan, b = nan
학습률 0.001,     5번째: 손실 27.05, w = 2.035, b = 0.069
학습률 0.001,   100번째: 손실 20.72, w = 1.996, b = -0.025
학습률 0.001,  5000번째: 손실 17.41, w = 2.141, b = -4.027
정답: w = 2.495, b = -13.758
```

nan = 숫자가 아님(not a number): 너무 커져서 계산이 무너진 것

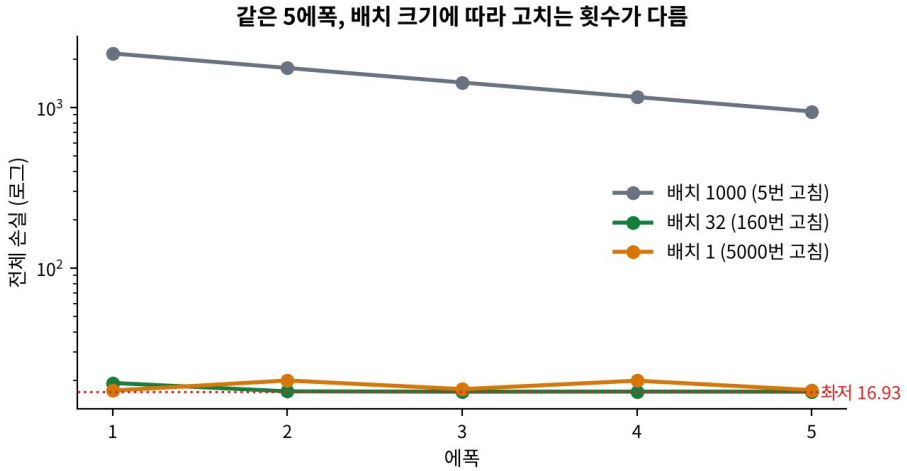
무슨 일이?

w 기울기 = 평균(2 × 오차 × x)
x가 18~34라 b 기울기보다 수십 배 큼
→ w는 튀고(0.01), b는 거의 안 움직임 (0.001)

해결: 표준화

x를 평균 0, 표준편차 1로 맞추면
w, b 기울기 크기가 비슷해짐
→ 학습률 하나로 둘 다 잘 내려감 (b07부터 표준화한 이유)

b24 미니배치 경사하강법: 일부만 보고 자주 고치기



배치 크기	특징
전체 (1000)	정확하지만 고치는 횟수가 적어 느림
32 (미니배치)	**자주 고치고 흔들림도 적음 → 실무 기본**
1	가장 자주 고치지만 손실이 출렁임

```
for _ in range(에폭):                          #에폭 = 전체 한 바퀴
    순서 = torch.randperm(len(x))              #매 에폭 섞기
    for 시작 in range(0, len(x), 배치_크기):
        고를 = 순서[시작:시작 + 배치_크기]
        손실 = ((w * x[고를] + b - y[고를]) ** 2).mean()
        손실.backward()                        #일부만 보고 손실
        with torch.no_grad():
            w -= 학습률 * w.grad
            b -= 학습률 * b.grad
            w.grad.zero_()
            b.grad.zero_()
        고친_횟수 += 1
```

```
배치 1000 |    5 번 고침 | 2171 1762 1430 1162  944
배치   32 |  160 번 고침 | 19.20 17.02 16.94 16.96 16.94
배치    1 | 5000 번 고침 | 17.20 19.89 17.57 19.83 17.34
도달할 수 있는 가장 낮은 손실: 16.93
```

b25 torch.autograd.grad: 기울기를 결과로 받기

```
x = torch.tensor(3.0, requires_grad=True)
y = x ** 3
# backward()와 달리 x.grad에 쌓지 않고 결과로 돌려줌
(dy_dx,) = torch.autograd.grad(y, x, create_graph=True)
#          create_graph=True: 기울기 계산도 기록 → 한 번 더 미분 가능
print("1번 미분 3x^2 :", dy_dx)        # 3 × 9 = 27
(d2y_dx2,) = torch.autograd.grad(dy_dx, x)    # 기울기의 기울기
print("2번 미분 6x   :", d2y_dx2)      # 6 × 3 = 18
print("x.grad        :", x.grad)

# 여러 텐서의 기울기를 한꺼번에
손실 = (w * 3.0 + b - 10.0) ** 2
기울기_w, 기울기_b = torch.autograd.grad(손실, [w, b])
```

```
1번 미분 3x^2 : tensor(27., grad_fn=<MulBackward0>)
2번 미분 6x   : tensor(18.)
x.grad        : None
손실의 w, b 기울기: -18.0 -6.0
```

	backward()	autograd.grad()
결과	x.grad에 쌓임	돌려받음
zero_ 필요	O	X
2번 미분	어려움	create_graph=True
주로	학습 루프	입력 민감도 (b26)

(dy_dx,) = …

결과가 튜플(묶음)로 오므로
쉼표를 붙여 하나만 꺼냄

b26 입력에 대한 기울기: 무엇에 얼마나 민감한가

```
def 모델(입력: torch.Tensor) -> torch.Tensor:
    """학습이 끝났다고 치는 모델 (예제용 식)."""
    기온, 습도 = 입력[0], 입력[1]
    return 2.5 * 기온 + 0.8 * 습도 + 0.05 * 기온 * 습도 - 15

오늘 = torch.tensor([30.0, 60.0], requires_grad=True)  # 입력에 표시!
예측 = 모델(오늘)
(민감도,) = torch.autograd.grad(예측, 오늘)
print(f"예측 판매량: {예측.item():.1f}개")
print(f"기온 1°C당 {민감도[0]:.2f}개")
print(f"습도 1%p당 {민감도[1]:.2f}개")
```

```
예측 판매량: 198.0개
기온 1°C당 5.50개 (손으로: 2.5 + 0.05 × 60 = 5.50)
습도 1%p당 2.30개 (손으로: 0.8 + 0.05 × 30 = 2.30)
```

무엇이 다를까

지금까지: w, b에 대한 기울기 (학습)
여기서는: 입력에 대한 기울기
→ '입력이 조금 바뀌면 예측이 얼마나 바뀌나'

어디에 쓰나

모델이 어떤 입력에 민감한지 설명
(이미지에서 중요한 픽셀 찾기 등)

확인 문제: 학습률·스케일·미니배치

21 학습률 1.0이면 x는?

```
for 학습률 in [1.0]:
```

22 학습률 0.5면 10·50·200번째 손실은?

```
for 학습률 in [0.5]:
```

23 표준화한 x로 학습률 0.01, 5000번이면?

```
x = (기온 - 기온.mean()) / 기온.std()
```

24 배치 크기 100이면 고친 횟수와 손실은?

```
for 배치_크기 in [100]:
```

25 y = x ** 4, x = 2이면 1번·2번 미분은?

```
x = torch.tensor(2.0, requires_grad=True)
```

26 기온 25, 습도 80이면 민감도는?

```
오늘 = torch.tensor([25.0, 80.0], ...)
```

확인 문제 정답: 학습률·스케일·미니배치

21 −5, 5, −5, 5, 5
배수 (1 − 2) = −1 → 영원히 왔다 갔다

22 모두 14.0772
표준화한 x에서는 0.5가 한 번에 도착하는 보폭

23 손실 14.077, w 12.656, b 52.500
발산 없이 정답에 도착

24 50번 고침
339.03 → 55.74 → 21.68 → 17.52 → 16.98

25 $4x^3 = 32$
$12x^2 = 48$

26 예측 211.5개
기온 1°C당 6.50 (2.5 + 0.05 × 80)
습도 1%p당 2.05 (0.8 + 0.05 × 25)

7

실습: 당뇨병 진행 정도 예측

b27 미니배치 학습 · b28 최소제곱법 정답과 비교

b27 ① 데이터 준비와 기울기 확인

```
from sklearn.datasets import load_diabetes
데이터 = load_diabetes(scaled=False)
X = torch.tensor(데이터.data, dtype=torch.float32)      # (442, 10)
y = torch.tensor(데이터.target, dtype=torch.float32)    # (442,)

torch.manual_seed(0)
순서 = torch.randperm(len(X))
학습_idx, 시험_idx = 순서[:350], 순서[350:]     # 350명 학습, 92명 시험
평균 = X[학습_idx].mean(dim=0)                   # 기준은 학습 데이터에서만
표준편차 = X[학습_idx].std(dim=0)
X = (X - 평균) / 표준편차

def 예측(X, w, b):
    return X @ w + b                # (n, 10) @ (10,) → (n,)

# 손 기울기 vs autograd
손_w = 2 * X_학습.T @ 오차 / len(X_학습)    # b07 공식을 10개 특성으로
손_b = 2 * 오차.mean()
```

```
기울기 일치: True True
```

항목	내용
입력 X	환자 442명 × 검사값 10개 (나이·성별·BMI·혈압·혈액 검사 s1~s6)
정답 y	1년 뒤 진행 정도 (25~346 점수)
w, b	w 10개 + b 1개 = 11개를 학습

X.T @ 오차

특성마다 '2 × 오차 × x' 평균을 한 번에
(4장 b17 행렬 곱, 전치)

b27 ② 미니배치로 학습하고 시험

```
학습률, 배치_크기 = 0.05, 32
for 에폭 in range(1, 101):
    섞기 = torch.randperm(len(X_학습))
    for 시작 in range(0, len(X_학습), 배치_크기):
        고를 = 섞기[시작:시작 + 배치_크기]
        손실 = ((예측(X_학습[고를], w, b) - y_학습[고를]) ** 2).mean()
        손실.backward()
        with torch.no_grad():
            w -= 학습률 * w.grad
            b -= 학습률 * b.grad
        w.grad.zero_()
        b.grad.zero_()

def rmse(X, y, w, b) -> float:
    """평균적으로 몇 점 틀리는지: 평균제곱오차의 제곱근."""
    return ((예측(X, w, b) - y) ** 2).mean().sqrt().item()
```

```
에폭   1: 학습 RMSE  72.23, 시험 RMSE  69.83
에폭   5: 학습 RMSE  54.18, 시험 RMSE  52.52
에폭  20: 학습 RMSE  54.14, 시험 RMSE  52.79
에폭 100: 학습 RMSE  54.06, 시험 RMSE  52.25
s1, s2 가중치: -25.44, 16.54
```

구조는 지금까지와 같음

손실 → backward → no_grad 고치기 → zero_
특성이 1개에서 10개로 늘었을 뿐

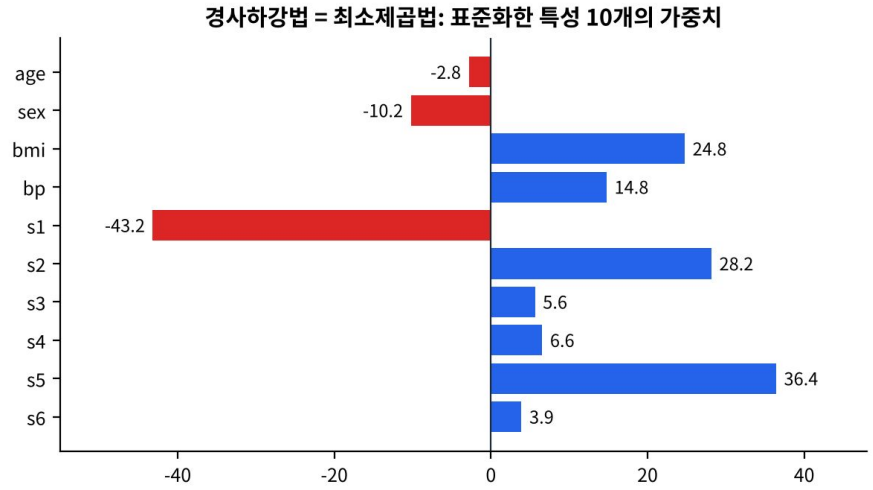
RMSE 읽기

시험 RMSE 52 = 평균 52점 정도 틀림
학습·시험이 비슷 → 외워 버린 것(과적합)은 아님

5에폭 이후

거의 그대로 → 직선 모델의 한계
더 좋아지려면 더 복잡한 모델 (6장)

b28 경사하강법 vs 최소제곱법 정답



s1(−43)과 s2(+28)처럼 서로 비슷한 검사값은 크게 반대 부호로 나뉨 → 가중치 크기만으로 '중요도' 판단은 주의

```
# 전체 데이터, 학습률 0.2, 5000번 (흔들림 없이 끝까지)
for _ in range(5000):
    손실 = ((예측(X_학습, w, b) - y_학습) ** 2).mean()
    ...   # backward, 고치기, zero_

# 한 번에 푸는 정답 (최소제곱법)
A = torch.cat([X_학습, torch.ones(len(X_학습), 1)], dim=1)
#   마지막 열 1은 b 자리
정답 = torch.linalg.lstsq(A, y_학습.unsqueeze(1)).solution.squeeze(1)
```

```
가장 큰 차이: 0.0004
학습 RMSE: 경사하강법 53.88, 최소제곱법 53.88
시험 RMSE: 경사하강법 52.35, 최소제곱법 52.35
평균만 찍었을 때 시험 RMSE: 80.35
```

결론

경사하강법만으로 정답과 같은 w, b에 도착
평균만 찍기(80.35)보다 28점 덜 틀림

확인 문제: 실습 당뇨병

27 **배치 크기를 32에서 350(전체)으로 바꾸면?**

```
학습률, 배치_크기 = 0.05, 350
```

28 **가중치 절댓값이 가장 큰 특성 두 개는?**

```
w.abs().argsort(descending=True)[:2]
```

확인 문제 정답: 실습 당뇨병

27

에폭 1 시험 RMSE 151.74 (아직 덜 배움)
에폭 100 시험 RMSE 52.30
에폭당 1번만 고쳐서 처음이 훨씬 느림

28

s1 (−43.2), s5 (36.4)
단, s1·s2처럼 서로 닮은 특성은
반대 부호로 나뉘어 커질 수 있음

추가 실습 (중급): 준비 코드

```
# b27과 같은 데이터 준비: X_학습, y_학습, X_시험, y_시험, 평균, 표준편차, 예측, rmse

def 학습(X_tr, y_tr, 학습률=0.05, 배치=32, 에폭=20):
    torch.manual_seed(1)                    # 정답과 같은 숫자가 나오도록
    w = torch.zeros(X_tr.shape[1], requires_grad=True)   # 특성 수만큼
    b = torch.zeros((), requires_grad=True)
    for 에폭_번호 in range(에폭):
        섞기 = torch.randperm(len(X_tr))
        for 시작 in range(0, len(X_tr), 배치):
            고를 = 섞기[시작:시작 + 배치]
            손실 = ((예측(X_tr[고를], w, b) - y_tr[고를]) ** 2).mean()
            손실.backward()
            with torch.no_grad():
                w -= 학습률 * w.grad
                b -= 학습률 * b.grad
            w.grad.zero_()
            b.grad.zero_()
    return w, b

w, b = 학습(X_학습, y_학습)
with torch.no_grad():
    print(rmse(X_시험, y_시험, w, b))      # 52.45
```

이렇게 진행

학습() 함수의 인자나 몸통을 조금씩 바꿔 가며 다섯 문제를 풀어 봄
먼저 결과를 예측해서 적고 실행

왜 함수로?

학습률·특성·에폭만 바꿔서 같은 조건으로 비교하려고
manual_seed(1)로 섞는 순서도 고정

X_tr.shape[1]: 특성 개수 (10개 또는 1개)

확인 문제: 추가 실습 (중급)

1 학습률을 0.001, 0.01, 0.05, 0.3으로 바꿔 20에폭 학습하면 시험 RMSE는?

```
학습(X_학습, y_학습, 학습률=0.001)
```

2 특성 하나만 쓰면? bmi, s5, age 각각 시험 RMSE (10개 모두는 52.45)

```
학습(X_학습[:, 2:3], y_학습)    # bmi만
# 2:3으로 잘라야 (350, 1) 모양 유지
```

3 학습() 안의 zero_ 두 줄을 지우고 에폭마다 시험 RMSE를 보면?

```
# w.grad.zero_()
# b.grad.zero_()
```

4 시험 첫 환자의 예측은? 원래 단위로 BMI·혈압이 1 오르면 예측은 얼마나 변할까?

```
원래 = X[시험_idx[0]].clone()
원래.requires_grad = True
p = 예측((원래 - 평균) / 표준편차, w, b)
(민감도,) = torch.autograd.grad(p, 원래)
```

5 50에폭 학습하며 에폭마다 시험 RMSE를 기록하면 가장 낮은 에폭은?

```
기록 = []    # 에폭이 끝날 때마다
with torch.no_grad():
    기록.append(rmse(X_시험, y_시험, w, b))
```

확인 문제 정답: 추가 실습 (중급)

1

0.001: 112.55 (아직 덜 배움)
0.01: 52.48 · 0.05: 52.45
0.3: inf (발산)

2

bmi 63.01 · s5 62.17 · age 77.49
하나만으로는 부족하고
age는 거의 도움이 안 됨

3

183.2 → 153.3 → 99.5 → 200.4 → 250.6 …
20에폭 뒤 477.8
기울기가 쌓여서 안정되지 못함 (b17)

4

예측 126.55 (실제 65)
BMI +1 → +5.68 · 혈압 +1 → +1.13
= 표준화 가중치 ÷ 표준편차
(BMI: 24.77 ÷ 4.36)

5

23에폭: 51.86 (가장 낮음)
50에폭: 52.37
오래 학습한다고 계속 좋아지지는 않음

추가 실습 정답 코드

```
# Q1
for 학습률 in [0.001, 0.01, 0.05, 0.3]:
    w, b = 학습(X_학습, y_학습, 학습률=학습률)
    with torch.no_grad():
        print(학습률, rmse(X_시험, y_시험, w, b))

# Q2  특성 번호: age 0, bmi 2, s5 8
for j in [2, 8, 0]:
    w, b = 학습(X_학습[:, j:j + 1], y_학습)
    with torch.no_grad():
        print(이름[j], rmse(X_시험[:, j:j + 1], y_시험, w, b))

# Q3  학습() 안의 zero_ 두 줄을 주석 처리하고
#     에폭 반복 끝에 시험 RMSE를 print
```

```
# Q4
w, b = 학습(X_학습, y_학습)
원래 = X[시험_idx[0]].clone()        # 표준화 전 원래 값
원래.requires_grad = True
p = 예측((원래 - 평균) / 표준편차, w, b)
(민감도,) = torch.autograd.grad(p, 원래)
print(p.item(), y_시험[0].item())
print(민감도[2].item(), 민감도[3].item())   # bmi, bp

# Q5  학습() 에폭 반복 끝에 추가 (에폭=50)
#     with torch.no_grad():
#         기록.append(rmse(X_시험, y_시험, w, b))
가장_좋은 = min(range(50), key=lambda i: 기록[i])
print(가장_좋은 + 1, 기록[가장_좋은])
```

생각해 볼 점

Q4: 원래 값에서 시작해 표준화까지 그래프에 넣으면 autograd가 '÷ 표준편차'까지 자동으로 미분해 줌
Q5: 시험 점수를 보고 에폭을 고르면 시험 데이터를 학습에 쓴 셈 → 실무에서는 검증 데이터를 따로 둠 (6장)

정리

01	**기울기**	조금 움직였을 때 변한 양 ÷ 움직인 거리. 방향마다 따로 (편미분)
02	**경사하강법**	새 값 = 값 − 학습률 × 기울기. 출발점에 따라 다른 골짜기
03	**자동 미분**	requires_grad → 계산 → backward → .grad. 손실은 숫자 하나
04	**연쇄 법칙**	단계별 지역 기울기를 끝에서부터 곱하며 거꾸로 전달
05	**함정**	grad는 쌓임(zero_) · 고치기는 no_grad · numpy는 detach · 제자리 연산 주의
06	**학습률 등**	크면 발산, 작으면 느림 · 입력은 표준화 · 미니배치 32 근처